Universidade de Coimbra
Faculdade de Ciências e Tecnologia

DEPARTAMENTO DE ENGENHARIA ELETROTÉCNICA E COMPUTADORES
Redes de Computadores

Class Assignment nº4
NS2 introduction
2024/2025

www.isi.edu/nsnam/ns/tutorial

---

1. Introduction

Simulation is a key element in the design of computer networks and in the evaluation of new network solutions and protocols. Through simulation it is possible to compare alternative solutions, avoiding the need to implement real evaluation scenarios, which are often impossible to carry out due to financial reasons or due technological reasons.
There are several commercial and free packages for the simulation of communication networks. There are simulators oriented to the study of specific technologies, others oriented to the evaluation of network models and algorithms. However, Network Simulator 2 (NS-2) has proved to be one of the important simulators in communication networks, given its increasing use. The richness of this simulator results from its open code and the voluntary work of many users and programmers spread all over the world.
Its generalist structure, which features a vast set of additional modules allowing it to cover any protocol layer in communication systems, and its dissemination make the NS-2 the simulator of choice of many academic and business professionals.

2) Objectives

In this lesson we intend to develop some Tcl scripts that simulate simple topologies, and evaluate their behavior in different scenarios using nam – The Network Animator.

3) Example 1 - Basic network

Below is a set of essential and generic instructions for any simulation to be performed using NS-2. In this section, instructions are also presented that will allow you to monitor (trace) the simulations.

3.1. Creating a scenario

To create a new simulation, it is necessary to define a Simulator object using the command:

```
set ns [new Simulator]
```

With this command, the *ns* variable becomes an instance of the Simulator object.

3.2. Simulation monitoring

Afterwards, it is necessary to create a file to store the simulation results. This file can later be used by *nam* to produce the final animations.

```
set nf [open out.nam w]
$ns namtrace-all $nf
```

In the first line of code, the file *out.nam* is opened for writing and the handler *nf* is assigned to it. In the second line, the simulator is informed to write all the simulation results in the respective file.

The next step consists of a sequence of commands, organized in the "end" procedure, so that, at the end of the simulation, the results file is closed and nam is invoked.

```
proc fim {} {
    global ns nf
    $ns flush-trace
    close $nf
    exec nam out.nam
    exit 0
}
```

Note: you should pay attention to the *nam* executable file path.

In the NS-2, events are scheduled using the at command. Like this,

```
$ns at 5.0 "fim"
```

invokes the end procedure after 5.0 seconds of simulation time.

3.3. Starting simulation

The simulation is started with the command:

```
$ns run
```

The presented set of instructions can be saved for later use, since it is a generic model for any simulation, and to which later commands for creating different topologies can be added.
Save the file with the name *example1.tcl*.

3.4. - Traffic between two nodes

This section intends to define a simple topology by generating traffic between two nodes.

3.4.1. Adding Two Nodes

Insert the following lines in the example1.tcl file before the line "`$ns at 5.0 "fim"`":

```
set n0 [$ns node]
set n1 [$ns node]
```

A node object is created using the `$ns node` command.

3.4.2. Establishing a connection

The next line defines a link between the two nodes.

```
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
```

A duplex connection between nodes n0 and n1 is defined with a bandwidth of 1 Mbps, with a delay of 10ms and with a DropTail type queue.
Save the file with the name example1.tcl.
To run the script call the command "`ns exemplo1.tcl`". At the end of the simulation, *nam* will be started and a topology similar to the one shown in figure 1 will be visible.

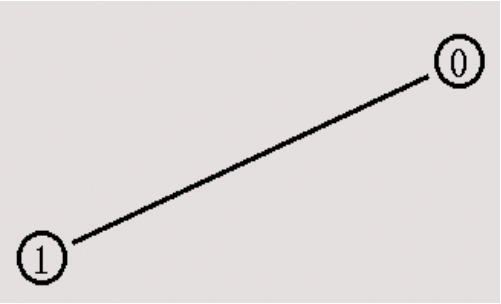


Figure 1

3.4.3. traffic generation

Once the topology has been defined, it is necessary to establish the traffic generating sources. For this example, we are going to insert an application on node *n0* to generate packages to *node n1*.
In NS-2, data is sent from one agent to another. Thus, it is necessary to create two agents, one that will send data from *node n0* and another that will receive data:

```
#Creates an UDP agent and attachs it to n0
set udp0 [new Agent/UDP]
$ns attach-agent $n0 $udp0

#Creates a CBR source traffic and attachs it to udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_  500
$cbr0 set interval_  0.005
$cbr0 attach-agent $udp0
```

A UDP agent is defined which is connected to *node n0*. In turn, a CBR (constant bit rate) traffic source will be defined to generate packets with a size of 500 bytes at a rate of 200 packets per second (1 in every 0.005 seconds).

3.4.4. traffic reception

In NS-2, a NULL agent is defined that will act as a data receiver.

```
# Create a Null agent and attachs it to node n1
set null0 [new Agent/Null]
$ns attach-agent $n1 $null0
```

3.4.5. Connections

Once the two agents have been defined, it is necessary to proceed with their connection.

```
$ns connect $udp0 $null0
```

3.4.6. transmission period

In a simulation, it is necessary to define the instants at which the CBR source starts transmitting data and stops that same transmission.

```
$ns at 0.5 “$cbr0 start”
$ns at 4.5 “$cbr0 stop”
```

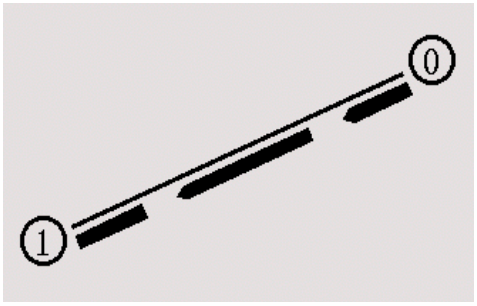


Figure 2

At this stage, you must save the file again. Afterwards, you should run the script you created and review the changes made to *nam* . You should see an image similar to the one shown in figure 2.
It is also possible to “click” with the mouse on any package in the *nam* window to monitor it. It is also possible to “click” on a given link to obtain information about its properties.

4) Example 2 - Behavior of a forwarder

In this example, four nodes will be defined, where one of them forwards the data sent by two of the nodes to the fourth node.

4.1. Adding 2 Nodes

The lines of code shown below establish a network with four nodes. The lines must be inserted in the *example1.tcl* file before the line *”$ns at 5.0 “fim”*”. Later, you should save the resulting file with the name *example2.tcl*.

```
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
```

4.2. Links establishment

The following code establishes the links between the three nodes:

```
$ns duplex-link $n0 $n2 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
```

If you try to run the script in *nam* you will find that the distribution of nodes is not the most desirable one. However, it is possible to control the arrangement of nodes and links by introducing some optional parameters. The following lines exemplify the topology layout control according to figure 3.

```
$ns duplex-link-op $n0 $n2 orient right-down
$ns duplex-link-op $n1 $n2 orient right-up
$ns duplex-link-op $n2 $n3 orient right
```

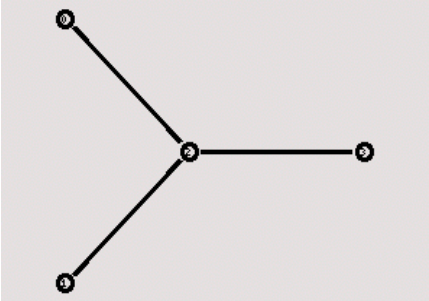


Figura 3

4.3. traffic generation

For this topology, two UDP agents will be created with CBR traffic sources connected to nodes *n0* and *n1*. A third *NULL* agent will be created, connected to node n3, which will receive the produced data.

```
#Creates a UDP agent and connects it to n0
set udp0 [new Agent/UDP]
$ns attach-agent $n0 $udp0

# Creates a CBR agent and connects it to udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_  500
$cbr0 set interval_  0.005
$cbr0 attach-agent $udp0

# Creates a UDP agent and connects it to n1
set udp1 [new Agent/UDP]
$ns attach-agent $n1 $udp1

# Creates a CBR agent and connects it to udp1
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_  500
$cbr1 set interval_  0.005
$cbr1 attach-agent $udp1

# Creates a NULL agent and connects it to n3
set null0 [new Agent/Null]
$ns attach-agent $n3 $null0
```

Both CBR agents must be connected to the NULL agent.

```
$ns connect $udp0 $null0
$ns connect $udp1 $null0
```

It is necessary to define the instants at which the CBR sources start and end the data transmission.

```
$ns at 0.5 "$cbr0 start"
$ns at 1.0 "$cbr1 start"
$ns at 4.0 "$cbr1 stop"
$ns at 4.5 "$cbr0 stop"
```

You can try running the script with the command "`ns exemplo2.tcl`".

4.4. Distinction of flows

As can be seen, the two flows are represented in the *nam* in black. In this way, it is not possible to distinguish which packets are discarded: if they are all packets from a single source, from two sources or if the discard is carried out equally.
To allow different streams to be distinguished, the NS-2 supports the use of different colors. This process can be performed through the following lines (placed before *"$ns at 5.0 "end""*):

```
$udp0 set class_ 1
$udp1 set class_ 2
```

```
$ns color 1 Blue
$ns color 2 Red
```

These last two lines must be placed immediately after the creation of the *Simulator* object.
If you run the simulation again and visualize the result with *nam* you will see that the flows are already differentiable. You will see that, on connection n2-n3, one of the flows will be preponderant, so it will be interesting to check what is happening in the outgoing queue of the router (*n2*). When viewing the simulation with *nam* you will see an image similar to figure 4

4.5. queue monitoring

The next line monitors the output queue for the connection from n2 to n3 (placed before *"$ns at 5.0 "end""*).

```
$ns duplex-link-op $n2 $n3 queuePos 0.5
```

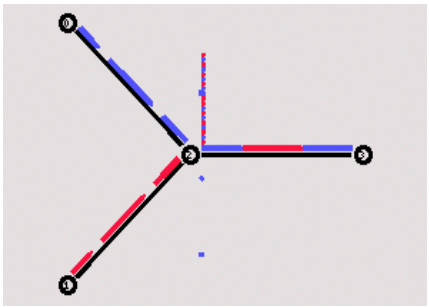


Figura 4

It is now possible to observe the packets that are in the queue and notice that only packets from one source are being discarded. Remember that a *DropTail* type queue was defined, which discards all packets that arrive when the queue buffer is full.
We will modify the type of queue management using fairer policies. Define an SFQ (stochastic fair queuing) queue and change the n2-n3 link definition to:

```
$ns duplex-link $n2 $n3 1Mb 10ms SFQ
```

If you run the simulation again, you will see that packets are discarded more evenly.

5) Exercise 1: Through some simple calculations it is also easily concluded that, for the previous exercise, the connection between nodes n2 and n3 has a lower bandwidth than the traffic generated for the period of 1 and 4 seconds. Prove then, performing simple calculations, that the connection between nodes n2 and n3 has a lower bandwidth than the traffic generated for the period of 1 and 4 seconds.

6) Network dynamics

In this example we will observe the behavior of a network in a situation of failure in one of the connections.

6.1. topology creation

The lines of code presented below define a network with seven nodes. The lines must be inserted in the *model.tcl* file before the line *"$ns at 5.0 "fim""*. Later you should save the file with the name *example3.tcl*.

```
for {set i 0} {$i < 7} {incr i} {
    set n($i)[$ns node]
}
```

In this exercise, the repetition cycle was used to facilitate the definition of nodes and their storage in the *n()* array. It is a more expeditious solution, which is often used when you want to create many nodes.
Next, the connections between the nodes will be made adopting a circular topology. The presented code allows each node to connect to the neighboring node. At the end, the last node will be connected to the first node.

```
for {set i 0} {$i < 7} {incr i} {
    $ns duplex-link $n($i) $n([expr ($i+1)%7]) 1Mb 10ms DropTail
}
```

The topology obtained will be similar to that shown in figure 5. If the appearance is quite different from this one, click on the "re-layout" button to approximate the appearance to that of the figure.

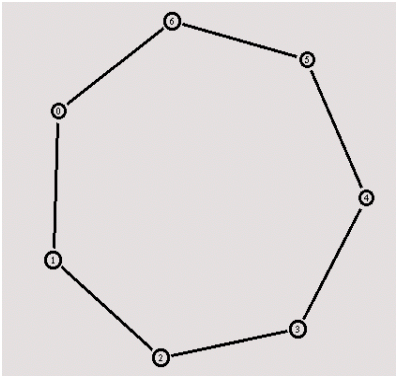


Figura 5

6.2. Traffic generation

In this step, traffic will be generated between two of the nodes. A UDP agent is created as a CBR type traffic source in node 0 and a NULL agent connected to node n3.

```
#Creates a UDP agent and connects it to n(0)
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0

# Creates a CBR agent and connects it to UDP
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_  500
$cbr0 set interval_  0.005
$cbr0 attach-agent $udp0

# Creates a NULL agent and connects it to n(3)
set null0 [new Agent/Null]
$ns attach-agent $n(3) $null0

$ns connect $udp0 $null0

$ns at 0.5 “$cbr0 start”
$ns at 4.5 “$cbr0 stop”
```

When viewing the simulation in *nam* you will see that the traffic will take the shortest path, using the connection that joins nodes n(0), n(1), n(2) and n(3) for that purpose.

6.3. Introduction of a fault in the topology

Now we will simulate a connection failure of nodes n(1) and n(2). For this purpose, the *rtmodel-at* command should be used, which can be used both in nodes and in connections. This command allows, through some parameters, to introduce events in the topology. Thus, the connection between nodes *n(1)* and *n(2)* will be brought down for one second.

```
$ns rtmodel-at 1.0 down $n(1) $n(2)
$ns rtmodel-at 2.0 up $n(1) $n(2)
```

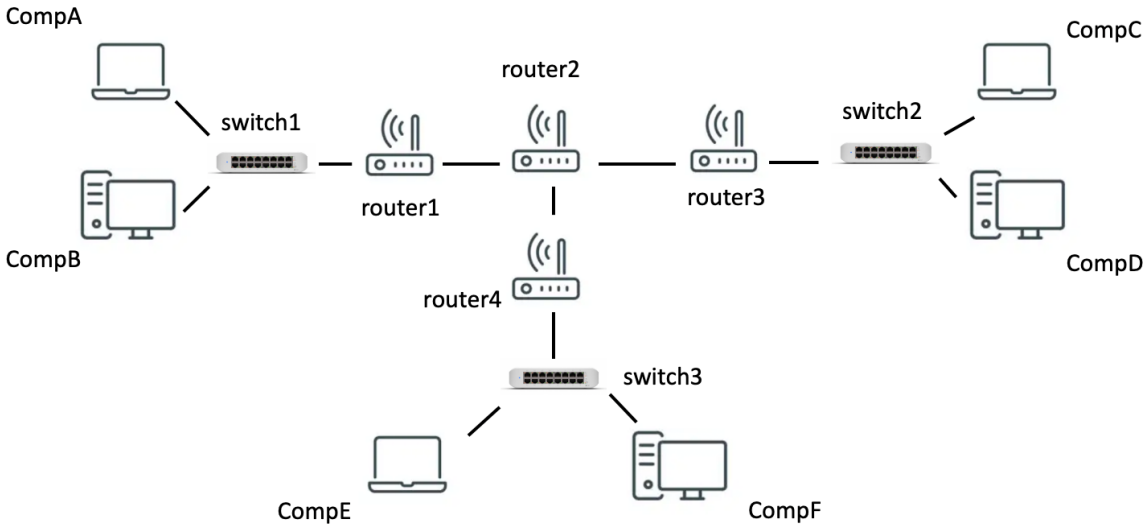
Running the script again you will be able to observe in *nam* that, when the connection is down, node *n(3)* stops receiving traffic. On the other hand, it appears that there is an alternative physical connection between nodes *n(0)* and *n(1)* but that this is not taken. Why?
To solve the problem, a dynamic routing algorithm should be adopted. To define a scenario with dynamic routing, add the line shown below, immediately after the definition of the *Simulator* object.

```
$ns rtproto DV
```

This line will define a routing based on the Distance Vector in the scenario. This routing protocol periodically sends (time defined by the *advertInterval* parameter) a path update message.
When the link between nodes *n(1)* and *n(2)* is interrupted, node *n(3)* stops receiving packets. Only after the routing table update message is sent does this node start receiving packets again, but this time through another path.

7) Exercise 2

After testing the previous examples, now you should implement a network according to the figure:



In this exercise, node *CompA* sends UDP blue packets to node *CompD*, and node *CompC* sends UDP blue packets to node *CompE*. There is also UDP red packets from node *CompB* to node *CompF*.
You should use 100 Mbps links with delays of 10ms. You can select any parameter that is missing.

Existing traffic should be:

```
$cbr… set packetSize_  500
$cbr… set interval_  0.005
```